

Deep Q-Networks for Single-Agent Decision-Making in Stochastic Environments: Minesweeper as a Benchmark

Ahmad Sayad - EN.705.643.8VL

Introduction

Q-learning is a fundamental reinforcement learning algorithm that seeks to learn the optimal action-value function, or Q-table, by iteratively updating values based on observed rewards. However, maintaining and updating a Q-table becomes computationally impractical for environments with large or continuous state-action spaces. To address this, Deep Q-Learning (DQL) approximates the Q-value function using neural networks, enabling scalability to more complex problems. Minesweeper, a classic grid-based puzzle game, serves as an ideal benchmark for testing reinforcement learning agents due to its inherent stochasticity: the placement of mines is random, and the outcomes of actions are often uncertain. This project investigates how effectively a Deep Q-Network (DQN) can perform in single-agent stochastic environments, using Minesweeper as a benchmark. By assessing the DQN's ability to navigate the game's challenges, the project provides insights into the potential and limitations of deep reinforcement learning in similar domains.

The results show that the DQN agent demonstrated significant progress in navigating the stochastic environment of Minesweeper, with the percentage of the board revealed serving as the primary performance metric. On average, the agent revealed between 53% and 66% of the board in its best-performing policies, while win rates peaked at 13.3% around the 250,000-policy mark. Additionally, the agent's average episode reward increased linearly during training up to the 250,000-episode mark, reflecting consistent learning progress. Despite this, performance began to decline in later stages, with both board coverage and win rates dropping, potentially due to overfitting or difficulties in balancing exploration and exploitation. Loss trends also became unstable towards the end of training. These findings demonstrate the DQN's ability to learn effective strategies and achieve substantial board coverage but also highlight its challenges in maintaining consistent performance in a probabilistic environment.

Datasets / Data the Agent Learns From

Unlike supervised learning approaches, where models are trained on predefined datasets, reinforcement learning agents like DQNs learn directly from interactions with the environment. In this case, the DQN selects what it believes to be a safe square (interaction) on the Minesweeper grid (environment). Each interaction provides the agent with a state (the current game grid), an action (a grid cell to reveal), a reward (based on the outcome of the action), and the next state (the resultant game grid from taking the action). These transitions are stored in a replay buffer as tuples, which serve as the agent's training data.

The DQN is trained by sampling mini-batches of transitions from the replay buffer. This approach allows the agent to reuse past experiences, breaking the correlation between sequential transitions. By optimizing the Q-value approximation using these samples, the agent incrementally learns a policy to maximize rewards despite the stochastic nature of the environment. There is no standard or static dataset because the state-action pairs and rewards are unique to the agent's exploration within the dynamic Minesweeper environment.

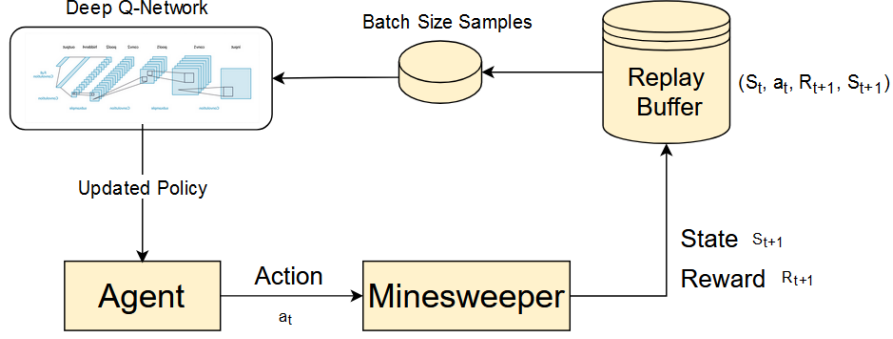


Figure 1: DQN Data & Architecture Overview

Methodology

Training the DQN involves having the agent interact with the Minesweeper environment for hundreds of thousands of episodes, each episode representing a single game of Minesweeper, to learn an optimal policy that maximizes cumulative rewards. Throughout these episodes, it stores each state, action, reward, and resultant state from every single action it took, into the replay buffer. Entries into the replay buffer then get sampled and are used to modify the weights of the DQN, resulting in a better approximation of the optimal action-value function. The three main entities that facilitate the learning are the environment, the replay buffer, and the DQN architecture. All three will be explained in detail throughout this section.

The Minesweeper environment serves as the foundation for training the DQN agent, defining the state representation, action space, and reward structure. It is modeled as a Markov Decision Process, defined by the tuple $\langle S, A, P, R, \gamma \rangle$. The state space S is represented as a grid $S \subseteq \mathbb{Z}^{H \times W}$, where H and W denote the dimensions of the minesweeper board. Each cell $s_t \in S$ holds one of the following values:

$$s_t = \begin{cases} -1, & \text{if the cell is unrevealed} \\ -2, & \text{if the cell contains a revealed mine} \\ k, & \text{revealed with } k \text{ adjacent mines, } k \in \{0, 1, \dots, 8\} \end{cases}$$

The action space A is defined as the set of all possible cell coordinates on the board:

$$A = \{(i, j) \mid 1 \leq i \leq H, 1 \leq j \leq W\}$$

Each action $a_t \in A$ corresponds to selecting a specific cell (i, j) to reveal. The state transition function $P(s_{t+1} \mid s_t, a_t)$ is the implementation of Minesweeper's rules: selecting a mine terminates the episode while selecting a safe tile reveals surrounding tiles recursively. The reward function $R(s_t, a_t)$ for each action a_t is designed to guide the agent towards uncovering safe tiles and avoiding mines:

$$R(s_t, a_t) = \begin{cases} c, & \text{if the action reveals } c \text{ new tiles} \\ -1, & \text{if the action is invalid} \\ -40, & \text{if the action reveals a mine} \\ +100, & \text{if the agent clears the board} \end{cases}$$

This structure encourages exploration of safe areas, penalizes repeated or dangerous actions, and heavily rewards completing the game. The choice of prioritizing long-term or short-term rewards in a given state is determined by the discount factor $\gamma = 0.8$.

Experience replay is a technique used to stabilize training in reinforcement learning by decorrelating successive updates. Transitions (s_t, a_t, r_t, s_{t+1}) encountered during the agent’s interaction with the environment are stored in a replay buffer D , implemented as a double-ended queue with a fixed capacity. This allows the agent to sample mini-batches of size B uniformly from the buffer during training, rather than relying only on the most recent transitions. This uniform sampling breaks the temporal correlations between consecutive experiences and ensures that the Q-network is updated using a diverse set of transitions.

The DQN approximates the action-value function $Q(s, a; \theta)$ using a convolutional neural network; this is because the input state is of the form $s_t \in \mathbb{Z}^{H \times W}$, meaning we can treat the grid as an image. The network architecture consists of two convolutional layers followed by two fully connected layers. The first convolutional layer consists of 32 filters with a kernel size of 3×3 , a stride of 1, and padding of 1, producing feature maps of the same spatial dimensions as the input grid. The second convolutional layer increases the depth to 64 filters, also using a 3×3 kernel, stride of 1, and padding of 1. The convolutional layers extract spatial features using operations of the form $h_1 = \sigma(W_1 * s_t + b_1)$ and $h_2 = \sigma(W_2 * h_1 + b_2)$, where $*$ denotes the convolution, W_i and b_i are learnable parameters, and σ is the ReLU activation function. The output from the convolutional layers, $h_2 \in \mathbb{R}^{64 \times H \times W}$ is flattened into a vector of size $64 \cdot H \cdot W$ for processing by the fully connected layers. The first fully connected layer reduces the dimensionality to 256 units with a ReLU activation function:

$$z = \phi(W_3 h_2 + b_3)$$

where ϕ is the ReLU activation. The second fully connected layer maps the 256-dimensional output to $|A| = H \times W$ Q-values, to compute the approximate action-value function:

$$Q(s, a; \theta) \approx W_4 z + b_4$$

This structure ensures that the output of the network corresponds to the Q-values for each possible action in A .

The training loop is what combines all three entities to train the DQN. As stated previously, the continuous interaction of the agent with the Minesweeper environment over hundreds of thousands of episodes is how the model is trained. The DQN is trained using experience replay and a target network to stabilize updates. Transitions (s_t, a_t, r_t, s_{t+1}) are stored in the replay buffer D , and mini-batches of size B are sampled randomly. For each sampled transition, the temporal difference target is computed as $y_t = r_t$ if the episode has terminated, or $y_t = r_t + \gamma \max_{a'} Q(s_{t+1}, a'; \theta^-)$ otherwise, where $Q(s_{t+1}, a'; \theta^-)$ is estimated using a lagged target network. The loss is calculated using a variation of the Huber loss function, known as Smoothed L1 Loss, defined as

$$\mathcal{L}(\theta) = \begin{cases} \frac{1}{2} \delta^2, & |\delta| \leq 1 \\ |\delta| - \frac{1}{2}, & |\delta| \geq 1 \end{cases}$$

where $\delta = y_t - Q(s_t, a_t; \theta)$. The network parameters θ are updated using the Adam optimizer with a learning rate $\eta = 1 \times 10^{-5}$.

The policy itself employs an ϵ -greedy exploration strategy, where the agent selects a random action with the following formula:

$$a_t = \begin{cases} \operatorname{argmax}_a Q(s_t, a; \theta), & \text{with probability } 1 - \epsilon \\ \text{random } a \in A, & \text{with probability } \epsilon \end{cases}$$

The exploration parameter ϵ decays exponentially according to $\epsilon = \max(\epsilon_{min}, \epsilon \cdot \text{decay})$. Additionally, every N training steps the target network parameters θ^- are updated to match the policy network θ . This stabilizes training by decoupling the Q-value estimation from the parameter updates. In the implementation of the DQN, $\epsilon_{min} = 0.1$, $\text{decay} = 0.998$, $N = 2000$, and the total number of games of Minesweeper (episodes) the model trained over was 300,000.

Testing & Experimentation

The performance of the trained DQN agent was evaluated by testing its ability to play Minesweeper on a standard 9x9 grid containing 10 mines. The primary metric used for evaluation was the average number of squares revealed by the agent per game. This metric reflects the agent's efficiency in uncovering safe areas of the grid while avoiding mines. The baseline for this experiment was set at revealing more than 50% of the board (approximately 41 squares) on average, which serves as a meaningful benchmark for the agent's success. The testing process consists of running the DQN agent through $N=1000$ independent games. At the start of each game, the Minesweeper environment is reset, and the agent begins with a blank board. The agent selects actions based on the trained policy network, which computes Q-values for all possible grid cells. The action corresponding to the highest Q-value is executed, and the agent continues playing until the game ends, either by revealing all safe tiles or hitting a mine.

For each game, the following data is recorded:

- **Squares Revealed:** The total number of safe tiles uncovered by the agent.
- **Percentage Revealed:** The proportion of the board successfully uncovered.
- **Game Outcome:** Whether the agent won (cleared the board) or lost (hit a mine).

After the test, the results are aggregated to compute their averages.

Experimental Results & Analysis

The training process for the DQN agent demonstrated a significant degree of effectiveness, as evidenced by the trends in the rewards and loss plots. The rewards plot showed a clear and consistent linear increase in the agent's average episode reward throughout training, up until approximately the 230,000-episode mark, indicating effective learning and improvement in the agent's decision-making capabilities. After this point, however, the rewards began to plateau and decline slightly, suggesting diminishing returns and potential overfitting.

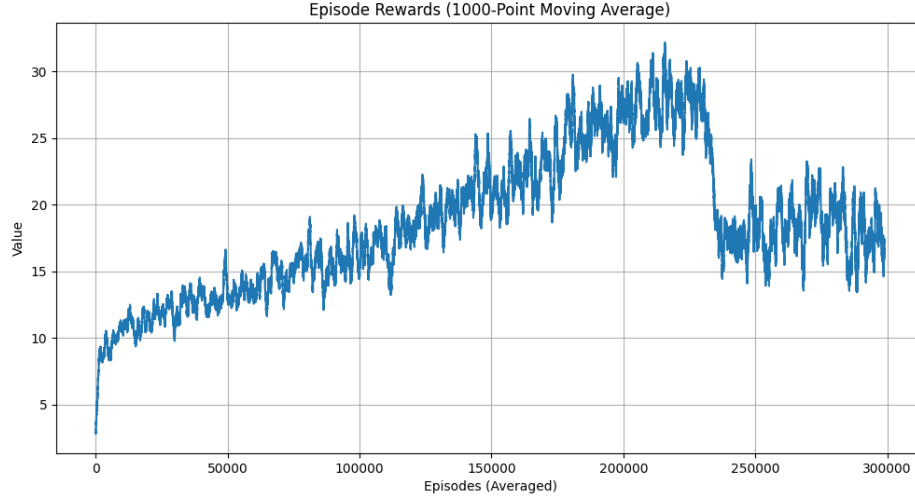


Figure 2: Cumulative Rewards Gained During Training Per Episode (1000-Point Average)

As shown in Figure 2, its peak cumulative reward on a 1000-point moving average was ~ 33 . Recall that the reward function was defined $R(s_t, a_t) = c$ for discovering c mines, and $R(s_t, a_t) = -40$ for hitting a mine. This implies that the best-performing policy was on average, clearing 73 out of the 81 cells on the Minesweeper grid; a 90.12% board-revealed percentage.

Similarly, the loss plot initially showed a steady decrease, reflecting the stabilization of the Q-value estimates, but later exhibited a sharp increase and fluctuation. This instability in the loss function toward the end of training suggests challenges in maintaining convergence, potentially due to overfitting or the agent's struggle to handle the increased complexity of decision-making in ambiguous situations.

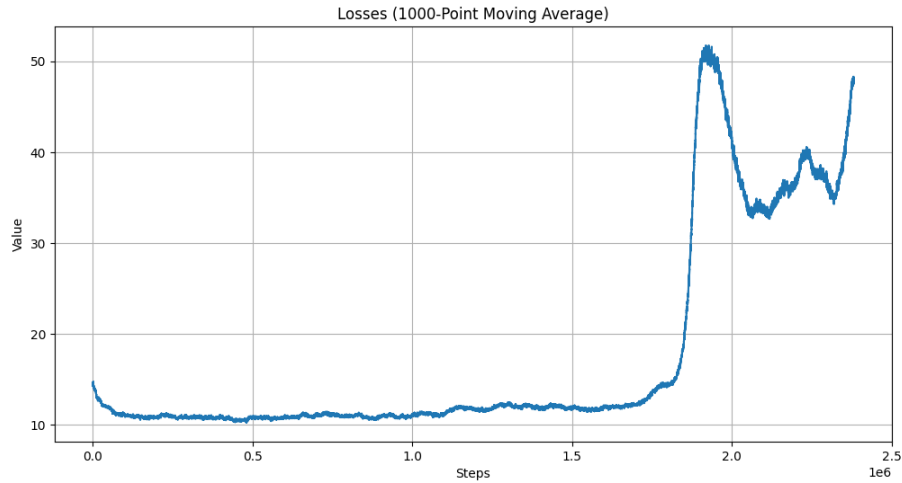


Figure 3: Losses Per Episode (1000-Point Moving Average)

Overall, the training was effective in enabling the agent to improve significantly during the early and middle phases, but its efficiency tapered off as training progressed, underscoring the importance of strategies to mitigate late-stage instability.

The following are the results achieved from the agent using different policies learned throughout training, tested over $N=1,000$ games of Minesweeper:

Policy Episode	Average Percent Revealed	Average Squares Revealed	Average Win Rate
50,000	66.19%	53.616	2.3%
150,000	65.18%	52.798	7.4%
250,000	66.19%	53.617	13.3%
300,000	52.58%	42.596	5.3%

Table 1: Testing Results, $N=1000$

The percentage of the board revealed ranged between 53% and 66% in the agent’s best-performing policies, reflecting its ability to clear a substantial portion of the board even under stochastic conditions. The win rate, while lower overall, peaked at 13.3% around the 250,000-policy mark, demonstrating that the agent could successfully navigate the game in some scenarios, particularly when ambiguity was resolved early. The relatively large increase in win rate compared to the stagnant average squares revealed shows that the model is learning what the optimal choices were to win the game – this is a very positive indicator of the feasibility of using DQNs in stochastic environments. The following are the aggregate results from all testing across all policies, plotted:



Figure 4: Testing Results Per Policy

The average number of squares revealed provided a more granular measure of the agent’s capabilities, with consistent improvement early in training but declining slightly in later policies, aligning the broader signs of overfitting or instability during training. The win rate steadily increasing despite not much relative difference in the number of squares being revealed also demonstrates that the DQN is learning optimal strategies to avoid mines. These metrics collectively suggest that while the agent excelled at partial board coverage (consistently breaking the benchmark result of revealing half the board), its ability to win the game outright was limited by its susceptibility to errors in ambiguous situations, reinforcing the challenge of probabilistic environments like Minesweeper.

Conclusion

The results of the study demonstrate the promise of DQNs for navigating stochastic and probabilistic environments like Minesweeper, with the agent achieving significant improvements in board coverage

and win rate throughout training. However, the plateauing and subsequent decline in performance metrics toward the later stages of training highlights the challenges of overfitting and instability in the loss function, particularly in environments with inherent ambiguity. These findings emphasize the need for strategies such as improved exploration techniques, dynamic reward adjustments, or hybrid architectures that incorporate probabilistic reasoning to handle uncertainty better. Future research could explore the robustness of DQNs in such environments through techniques like reward shaping, hierarchical reinforcement learning, or the incorporation of auxiliary objectives to address ambiguity. Additionally, exploring hybrid models that combine deep learning with probabilistic reasoning, or integrating transfer learning to leverage knowledge from related tasks, could further improve agent performance. These avenues offer promising directions to refine the use of DQNs in complex, real-world scenarios requiring adaptive and probabilistic decision-making.